Контест 8 · Безопасность

Неделя 16 · темы недель 15–16 и всего T2: подписи и пароли, ввод от игроков, лимиты, античит, токены, права агента. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Подписи и пароли

A1. `sign(secret, payload)` и `verify(secret, payload, sig)` — подпись словаря: HMAC-SHA256 (модули `hmac`, `hashlib`) от канонического JSON — ключи по алфавиту, без пробелов (`sort_keys=True, separators=(",", ":")`, `ensure_ascii=False`), ответ в hex. `verify` сравнивает через `hmac.compare_digest`.

```
Ввод:  sig = sign("s3cret", {"user": 42, "score": 120})
       verify("s3cret", {"score": 120, "user": 42}, sig)
       verify("s3cret", {"user": 42, "score": 9999}, sig)
Вывод: sig = "6f953c8dd277e649e1930eb4ef5dd8307298821a2ab92c518b30c7bb1b8f92d4"
       True
       False
```

A2. `hash_password(password, salt, iterations=100_000)` — `hashlib.pbkdf2_hmac("sha256", …)`, строка `"pbkdf2_sha256$итерации$соль_hex$хеш_hex"`. `check_password(password, stored)` — `True` / `False`; испорченная строка — `False`, без исключений.

```
Ввод:  h = hash_password("qwerty", bytes.fromhex("a1b2c3d4"), 1000)
       check_password("qwerty", h), check_password("qwertY", h)
       check_password("x", "bad")
Вывод: h = "pbkdf2_sha256$1000$a1b2c3d4$"
           "3702611b3c803fe657bd2829f1a2cd358e57e9e2015c32e48871cb322d6bdf29"  # слитно
       True, False
       False
```

A3. `naive_equal_steps(a, b)` — наивное сравнение строк: разной длины — `(False, 0)`; иначе слева направо до первого несовпадения. Ответ — `(равны ли, сколько пар символов сравнили)`. По времени такого сравнения можно угадать, сколько первых символов совпало.

```
Ввод:  naive_equal_steps("hunter2", "hunter3"), naive_equal_steps("abc", "abcd"),
       naive_equal_steps("pin7", "pin7")
Вывод: (False, 7), (False, 0), (True, 4)
```

A4*. `timing_attack(oracle, alphabet, length)` — `oracle(guess)` возвращает то же, что A3 для секрета длины `length`. Восстановите секрет, вызвав `oracle` не больше `len(alphabet) · length` раз. Объясните, почему `compare_digest` от этого защищает.

```
Ввод:  secret = "k3y9"; oracle = lambda g: naive_equal_steps(secret, g)
       timing_attack(oracle, "0123456789abcdefghijklmnopqrstuvwxyz", 4)
Вывод: "k3y9"                 # 118 вызовов вместо 36⁴ = 1 679 616
```

B. Ввод от игроков

B1. `safe_name(name)` — имя для рейтинга: `unicodedata.normalize("NFC", …)`, пробелы по краям убрать, несколько подряд — один. Длина от 2 до 16, символы — буквы и цифры любого алфавита (`isalnum`), `_`, `-`, пробел. Имена `admin`, `system`, `bot` в любом регистре заняты. Нарушение — `ValueError`.

```
Ввод:  safe_name("  Аня  Ли "), safe_name("Боря_2010")
       safe_name("Ann<script>"), safe_name("ADMIN")
Вывод: "Аня Ли", "Боря_2010"
       ValueError, ValueError
```

B2. `render_rating(rows)` — HTML-список рейтинга `(имя, очки)`: по убыванию очков, при равенстве — по имени. Экранирование своё (`escape_html`): `& < > " '` → `& < > " '`. Строки через `"\n"`.

```
Ввод:  render_rating([("bob", 50), ("<img src=x>", 120), ("ann", 50)])
Вывод: <ol>
       <li><b>&lt;img src=x&gt;</b> — 120</li>
       <li><b>ann</b> — 50</li>
       <li><b>bob</b> — 50</li>
       </ol>
```

B3. `find_sql_injection(code)` — номера строк (с 1), где у `execute(` или `text(` первый аргумент — f-строка или строковый литерал, к которому применяют `%`, `+` или `.format(`. Параметры вторым аргументом — это нормально.

```
Ввод:  1 rows = db.execute(f"SELECT * FROM players WHERE name = '{name}'")
       2 db.execute(text("SELECT * FROM players WHERE id = :id"), {"id": pid})
       3 db.execute("DELETE FROM matches WHERE id = " + mid)
       4 q = text("SELECT %s" % col)
       5 cur.execute("SELECT * FROM t WHERE a = %s", (a,))
Вывод: [1, 3, 4]
```

C. Лимиты

C1. `fixed_window(times, limit, window)` — фиксированные окна `[k·window, (k+1)·window)`, в каждом не больше `limit` разрешённых. Ответ — разрешён ли каждый запрос. Найдите в примере, сколько запросов прошло за 13 секунд.

```
Ввод:  fixed_window([0, 10, 50, 59, 60, 61, 62, 119, 125], 3, 60)
Вывод: [True, True, True, False, True, True, True, False, True]
```

C2. `gcra(times, period, burst)` — алгоритм GCRA: число TAT сначала 0. Запрос в момент `t`: `cur = max(TAT, t)`; если `cur − t > period · (burst − 1)` — отказ, TAT не меняется; иначе разрешить и `TAT = cur + period`.

```
Ввод:  gcra([0, 0, 0, 0, 100, 250, 250, 1000], 100, 3)
Вывод: [True, True, True, False, True, True, False, True]
```

C3. `fail2ban(failures, threshold, window, ban)` — неудачные входы `(ts, ip)` (отсортируйте по `ts`). Набралось `threshold` неудач в `(t − window, t]` — бан `(ip, t, t + ban)`. Неудачи во время бана не считаем, после — счёт с нуля.

```
Ввод:  A, B = "1.1.1.1", "2.2.2.2"
       fail2ban([(0, A), (5, A), (7, B), (9, A), (20, A),
                 (40, A), (80, A), (85, A), (88, A)], 3, 10, 60)
Вывод: [("1.1.1.1", 9, 69), ("1.1.1.1", 88, 148)]
```

C4*. `clusters(logins)` — входы `(user, ip, device)`. Пользователи связаны, если у них общий ip или общее устройство — в том числе через цепочку других. Найдите группы от двух человек (мультиаккаунты для накрутки): внутри по имени, группы по возрастанию. IP и устройство — разные пространства имён.

```
Ввод:  clusters([("ann", "ip1", "phoneA"), ("bob", "ip2", "phoneA"),
                 ("cat", "ip2", "pc1"), ("dan", "ip3", "pc2"),
                 ("eve", "ip4", "pc3"), ("fay", "ip4", "pc4")])
Вывод: [["ann", "bob", "cat"], ["eve", "fay"]]
```

D. Античит

D1. `replay_score(events, claimed)` — сервер сам пересчитывает счёт змейки: `eat` +10, `gold` +50, умноженные на серию. Серия растёт на 1 (не больше 3), если с прошлой еды было не больше двух `tick`, иначе снова 1. После `die` события не считаем; неизвестное событие — `ValueError`. Ответ — `(счёт, совпал ли с claimed)`.

```
Ввод:  replay_score(["eat", "tick", "eat", "tick", "tick", "gold",
                     "tick", "tick", "tick", "eat", "die", "gold"], 250)
Вывод: (190, False)            # 10 + 20 + 150 + 10
```

D2. `seq_check(messages)` — сообщения `(игрок, seq)`. Для каждого: `"dup"` — seq не больше последнего принятого у этого игрока (повтор, не принимаем); `"gap"` — больше ожидаемого (принимаем); иначе `"ok"`. Первый ожидаемый — 1.

```
Ввод:  seq_check([("ann", 1), ("ann", 2), ("bob", 1), ("ann", 2),
                  ("ann", 5), ("ann", 3), ("bob", 2)])
Вывод: ["ok", "ok", "ok", "dup", "gap", "dup", "ok"]
```

D3. `fog_of_war(units, player, radius)` — что отправить игроку: его юниты `(владелец, x, y)` и чужие на расстоянии Чебышёва (`max(|dx|, |dy|)`) не больше `radius` от любого его юнита. Порядок — как во входе.

```
Ввод:  fog_of_war([("ann", 0, 0), ("bob", 2, 1), ("bob", 5, 5),
                   ("cat", 9, 9), ("ann", 8, 8)], "ann", 2)
Вывод: [("ann", 0, 0), ("bob", 2, 1), ("cat", 9, 9), ("ann", 8, 8)]
```

E. Проект и алгоритмы

E1. Чек-лист безопасности команды (`docs/security.md` , семинар 15): покажите три закрытых пункта и как их проверить — запросом, тестом или командой.

E2. Демо: попробуйте подделать счёт или ход своей игры через DevTools или `curl` на staging — сервер должен отказать. Покажите строку лога с причиной отказа, без секретов.

E3. Отчёт агента-аудитора: одна верная находка, одна ложная и одна, которую агент пропустил, а люди нашли.

E4 · босс. Мини-JWT на HMAC-SHA256. base64url без `=` в конце, JSON компактный с ключами по алфавиту. За каждую часть свой плюс.

а) `encode(payload, secret, kid="k1")` — `"заголовок.данные.подпись"`, заголовок `{"alg": "HS256", "kid": kid, "typ": "JWT"}`, подпись — HMAC-SHA256 от строки `"заголовок.данные"`.

```
Ввод:  encode({"sub": 42, "iat": 1000, "exp": 1600}, "key-one", "k1")
Вывод: "eyJhbGciOiJIUzI1NiIsImtpZCI6ImsxIiwidHlwIjoiSldUIn0."
       "eyJleHAiOjE2MDAsImlhdCI6MTAwMCwic3ViIjo0Mn0."
       "vjgFsc7SBxsA7sciCKmX3excOGUsbko006pJIQBlrhk"          # три части слитно
```

б) `decode(token, keys, now, leeway=0)` — `keys`: `kid → секрет`. Проверки по порядку, ошибка — `ValueError` с текстом: `"format"`, `"alg"` (только `HS256`, никакого `none`), `"kid"`, `"signature"`, `"expired"` (`now > exp + leeway`), `"not_yet"` (`now < nbf − leeway`). Ответ — данные.

```
# T — токен из части а)
Ввод:  decode(T, {"k1": "key-one"}, 1200)
       decode(T, {"k1": "key-one"}, 1700), decode(T, {"k1": "wrong"}, 1200)
Вывод: {"exp": 1600, "iat": 1000, "sub": 42}
       ValueError("expired"), ValueError("signature")
```

в) `refresh(token, keys, current_kid, now, ttl, max_age)` — новый токен с теми же данными, `iat = now`, `exp = now + ttl`, подписанный ключом `current_kid` (ротация ключей). Истёкший — ошибка `decode`; нет `iat` — `"iat"`; `now − iat > max_age` — `"too_old"`.

```
Ввод:  r = refresh(T, {"k1": "key-one", "k2": "key-two"}, "k2", 1500, 600, 3600)
       decode(r, {"k2": "key-two"}, 1500)
Вывод: {"exp": 2100, "iat": 1500, "sub": 42}
```

E5*. `audit_policy(rules, calls)` — права агента. Правила `(allow|deny, шаблон)` по порядку, решает первое совпавшее, не совпало ни одно — `deny`. Шаблон — `fnmatch.fnmatchcase`, плюс фигурные скобки `{a,b}` (не вложены, их может быть несколько). Ответ — `(решения по вызовам, номера правил с 0, которые ни разу не решили)`.

```
Ввод:  rules = [("deny", "shell:git push*"), ("allow", "shell:git {status,diff,log}*"),
                ("allow", "shell:{pytest,ruff}*"), ("deny", "read:*.env"),
                ("allow", "read:*"), ("allow", "shell:git commit*")]
       audit_policy(rules, ["shell:git status", "shell:git push origin main",
                            "shell:pytest -q", "read:src/app.py", "read:deploy/.env",
                            "http:pypi.org", "shell:rm -rf /"])
Вывод: ([True, False, True, True, False, False, False], [5])
```